as the root. They run as ordinary user *xvm* with an appropriate privilege. This way, OpenSolaris complies with security out-of-the-box.

DTrace

We can trace the IO path *read (2)* takes in a domU using *dtrace (1M)*. Knowing that the entry point into a block driver is the strategy routine, let us go after *xdf_strategy()*, the strategy routine of the block front-end driver:

```
surya@opensolaris:~$ pfexec dtrace -n xdf::entry'{@[stack()]=count()}'
dtrace: description 'xdf_strategy:entry' matched 1 probe
```

Meanwhile, from a domU terminal, *cat (1)* a text file; then come back to the DTrace terminal and Ctrl+C it; you will find a stack similar to the one below:

```
genunix`bdev_strategy+0x75
genunix`ldi_strategy+0x59
zfs`vdev_disk_io_start+0xd3
zfs`zio_vdev_io_start+0x17d
zfs`zio_execute+0xa0
zfs`zio_nowait+0x5a
zfs`vdev_mirror_io_start+0x148
zfs`zio_vdev_io_start+0x1ba
zfs`zio_execute+0xa0
zfs`zio_nowait+0x5a
zfs`arc_read_nolock+0x82a
zfs`arc_read+0x75
zfs`dbuf_read_impl+0x159
zfs`dbuf_read+0xfe
zfs`dmu_buf_hold_array_by_dnode+0x1bf
zfs`dmu_buf_hold_array+0x73
zfs`dmu_read_uio+0x4d
zfs`zfs_read+0x19a
genunix`fop_read+0x6b
genunix`read+0x2b8
  1
```

This stack gives you enough details on how the read call is directed through different ZFS routines before *xdf* is invoked. Knowing that, ZFS calls the *zfs_interrupt* routine whenever an IO finishes. Let's trace that call too—we'll get a stack trace similar to the following:

```
zio_interrupt:entry
zfs`vdev_disk_io_intr+0x6b
genunix`biodone+0x84
xdf`xdf_io_fini+0x93
xdf`xdf_intr_locked+0x78
xdf`xdf_intr+0x2b
unix`av_dispatch_autovect+0x7c
unix`dispatch_hardint+0x37
unix`switch_sp_and_call+0x13
```

This gives us an understanding of what happens when the notification comes in upon IO completion.

Similarly, we can trace the IO path in dom0 as well, to see how the Xen back-end driver talks to the actual driver by going after the *xdb_biodone* call. Likewise, we can observe the network IO paths too—albeit by tracing STREAMS routines.

There is a new DTrace provider, *xdt*, which monitors hypervisor events like vCPU switching. All the probes associated with it can be found by using *dtrace -l -P xdt*.

After observing o/p from the above DTrace command, we can understand why the status of a domU, as reported by *virsh list*, is blocked—because the vCPU of osol domU is not currently running on any physical CPU.

We can use */usr/lib/xen/bin/xenstore-snoop* to snoop on the interaction between dom0 and domU via XenStore. We can run this command in a dom0 terminal, and from another dom0 terminal just run *virsh dumpxml osol* and see what *xenstore-snoop* reports.

Similarly, run the *format* command from a domU terminal and see what the above snoop reports. For the curious, this command is implemented using the DTrace PID provider. This command comes in handy to keep an eye on XenStore if we perceive any slowness in the system.

Crash file support

In case the hypervisor panics, OpenSolaris dom0 is extended to collect the core of the whole system in which Xen itself will be available as a kernel module. More information on this is available at *http://blogs.sun.com/nilsn/entry/debugging_an_xvm_panic*.

Secondly, if we find that a PV OpenSolaris guest is hung (with no response on console and not reachable over the network), we could collect the core of the domU from dom0 as follows, and use *mdb (1)* on the core file to investigate further:

```
surya@opensolaris:~$ pfexec xm dump-core osol
Dumping core of domain: osol ...
surya@opensolaris:~$ pfexec mdb /var/xen/dump/2009-0701-1528.42-
osol.3.core
Loading modules: [ unix genunix specfs dtrace mac ... logindmux  ufs
nsmb sppp mpt emlxs ]
>
```

We have just scratched the surface of the magic that can be done with OS2009.06 as dom0. Explore more at *www.opensolaris.org/os*. END

By: Surya Prakki

Surya debugs OpenSolaris kernel, both for living and for passion, and relies a lot on the lifeline called dtrace(1M). He occasionally blogs at blogs.sun.com/sprakki.